Information Retrieval using LLM

Independent Study for Fall 2024

Shuai Yang

Background and Model Introduction

My study was derived from a research paper titled “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”[1] and applied to the TREC ToT[2] project and 4Chan Elections project.This research focuses on the Retrieval-Augmented Generation (RAG) model, which combines the strengths of retrieval-based and generation-based approaches to handle knowledge-intensive tasks in natural language processing (NLP). The Retrieval-Augmented Generation (RAG) model is a technique that enhances the capabilities of generative AI models by incorporating information retrieval. Essentially, it combines the strengths of retrieval-based and generation-based approaches to handle knowledge-intensive tasks in natural language processing (NLP).The purpose of my projects was to apply the RAG model to specific datasets within particular domains.Two main applications were explored: the TREC ToT[2] project and the 4Chan Elections project. First, the TREC ToT project involved a movie dataset. In this context, the RAG model was used to retrieve relevant information, such as movie titles, from a given context based on a user's query. This demonstrates how the RAG model can effectively leverage external knowledge to provide accurate and contextually appropriate information.Second, the 4Chan Elections project focused on social media data. Here, the RAG model was employed to summarize or answer queries about posts on specific topics, such as election discussions, within the social media platform. This application showcases the RAG model's ability to handle unstructured data and generate coherent summaries or responses based on the content of the posts.Overall, these projects illustrate the versatility of the RAG model in addressing different types of knowledge-intensive tasks across various domains, from structured movie datasets to dynamic social media environments.

First, I’d like to introduce the RAG model by providing an overview of its architecture (Figure 1). At a high level, the RAG model consists of two main components: a retriever and a generator. This hybrid model combines parameterized memory with non-parametric (i.e., retrieval-based) memory to access and utilize external knowledge, thus expanding and enriching the knowledge base directly.

The RAG model uses the input sequence x to retrieve multiple text documents z. During the retrieval process, q(x), a query representation is produced by a query encoder. Query encoder and the query representations are both based on BERT. Now, it becomes a Maximum Inner Product Search (MIPS) problem that generates a list of k documents z with highest probability

p(z|x). The technique I used for retrieval is a pre-trained bi-encoder architecture from Dense Passage Retrieval (DPR)(Figure 2).

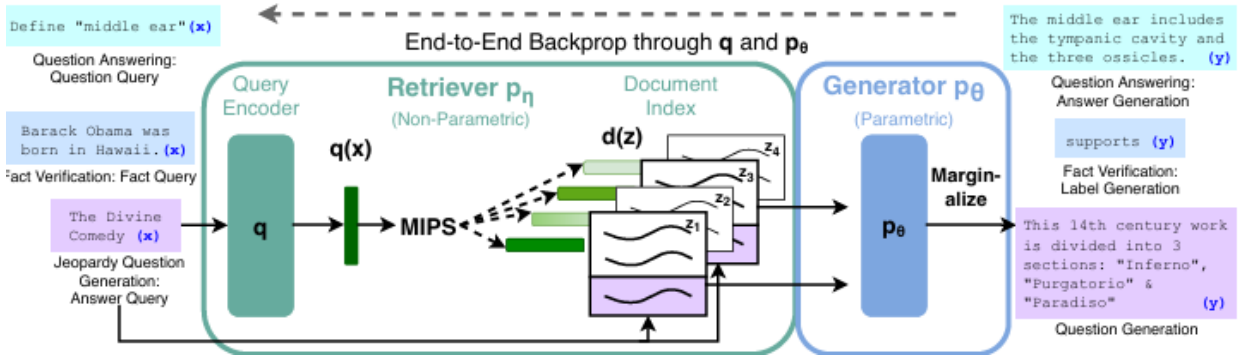


Figure 1 RAG Architecture[1]

As shown in Figure 2, DPR employs dense vector representations to capture the semantic meaning of both questions and passages. That is a bi-encoders architecture, one is Query/Question Encoder and the other is Document/Passage Encoder. Once encoded vectors for queries and passages are ready, I use cosine similarity for measurement to rank the documents/passages by relevance. One thing worth mentioning is that the DPR is pre-trained by minimizing distance of positive pairs and maximize distance of negative pairs.

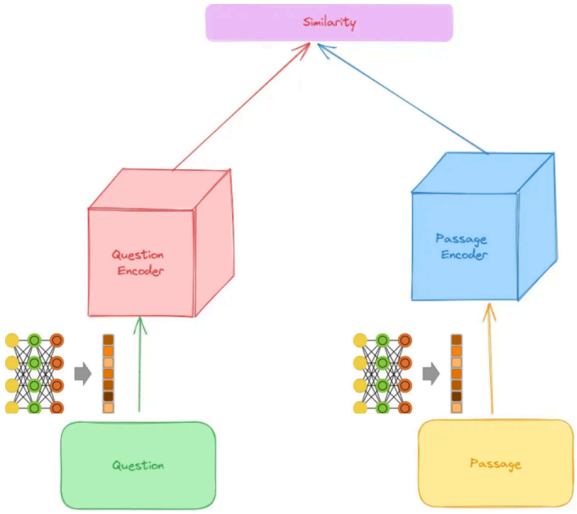


Figure 2 DPR Architecture [6]

Generator is the other key component of the RAG model. It conditions on the retrieved documents to generate an informed response. The retrieved documents, denoted as $d(z)$, are not just used for ranking but directly influence the text generation. Once a retriever returns multiple relevant documents d_1, d_2, ..., d_N, each document is treated as additional context for generation. The response is generated conditioned on each retrieved document. Instead of selecting a single document, RAG marginalizes over multiple retrieved documents.

This means that for every possible response y, , we sum over all retrieved document z, weighting each document's contribution based on how relevant it is.

Project Implementation and Results

- ToT project

The ToT identification task is straightforward - given an input ToT query/request, output a ranked list of documents/items. Each item can be either a movie, a landmark, or a celebrity, which are the domains of this year's project. For each query, the run should return a ranked list of 1000 Wikipedia page ids/document ids and have the correct item be ranked as high as possible. The runs will be evaluated using IR metrics, such as success@k, Discounted Cumulative Gain (DCG) and reciprocal rank. Success@k metric measures the proportion of successful queries where the relevant document is ranked within the top-k results. DCG measures the usefulness or gain of a document based on its position in the result list. The gain is discounted logarithmically, so higher-ranked documents contribute more to the overall score. Reciprocal Rank metric evaluates the rank position of the first relevant document for a given query. It is calculated as the reciprocal of the rank (1/rank) of the first relevant document. Higher scores indicate better performance, as the relevant document appears earlier in the result list.

- 4Chan Election project

The 4chan Election project is similar in retrieval phase but in a social media domain for the Election topic and has an additional generation phase that responses the input query. Specifically, the input items are the posts/comments on 4chan's board /pol/, which is a general discussion particular for politics. The output is a model response given by a prompt as query.

The implementations of the above two projects were deployed on a lab remote server with GPU available and launched in Notebooks.

a. Datasets

For the ToT project, I was given a large corpus (JSONL) with more than 3.2M entries. The corpora is subsets of Wikipedia pages relating to the domains mentioned later. As shown in Figure 3, I was also given train, dev, and test queries for training, validation, and testing purposes. Note that our train and dev queries come only from the movie domain while our test queries will include a combination of movies, landmarks. Figure 4 and Figure 5 show an example document and an example query for the ToT project.

For the 4Chan project, I used January's data as our corpus. The corpus contains 3.1M entries and 35 attributes. It was saved in a pickle file loading from AWS S3 buckets. The corpus

example is shown in Figure 6. There's no absolute ground truth dataset available but a csv file extracted from general /pol/ board regarding Trump provided by Jeremy in Figure 7.

Description	Link	# entries
corpus, train, dev1, dev2 (single zip)	TREC-ToT.zip	-
corpus (JSONL)	corpus.jsonl.zip	3,185,450
train queries (JSONL) & qrel.txt	train.zip	150
dev1 queries (JSONL) & qrel.txt ('23 dev set)	dev1.zip	150
dev2 queries (JSONL) & qrel.txt ('23 test set)	dev2.zip	150
test queries (JSONL)	test-2024.zip	600

Figure 3 ToT Datasets Overview


```
{
  "doc_id": "846",
  "text": "The Museum of Work (\"Arbetets museum\") is a museum located in Norrk\u00f6ping, Swede
  "sections": [
        {
        "start": 0,
        "end": 798,
        "section": "Abstract"
        },
        {
        "start": 798,
        "end": 1620,
        "section": "Overview"
        },
        {
        "start": 1620,
        "end": 3095,
        "section": "Exhibitions"
        },
        {
        "start": 3095,
        "end": 3371,
        "section": "The history of Alva"
        },
        {
        "start": 3371,
        "end": 3824,
        "section": "Industriland"
        },
        {
        "start": 3824,
        "end": 4371,
        "section": "Framtidsland (Future country)"
        },
        {
        "start": 4371,
        "end": 4761,
        "section": "EWK \u2014 The Center for Political Illustration Art"
        }
  ],
  "title": "Museum of Work",
  "wikidata_id": "Q6941060"
}
```


Figure 4 An ToT Example Document

I couldn't have been older than 4, so this was around 2002. I watched a movie with my parents (or so I thought) and despite never watching it again, it became my favorite. It centered around a middle aged plot man who went on some kind of adventure and turned into a fish. I also think I recall him visiting a uncertainty school of some sort? It seemed like a slightly old movie, but it was in color and began with real actors visua and changed to animation. For weeks after I saw this movie I told my parents about it, but they insisted it was a dream so I let it go. Does anyone know what this movie is?

Figure 5 An ToT Example Query

	com	country	country_name	id	name	no	now	resto	time	board_flag	...
0	>&g...	NO	Norway	nNk5Ps3S	Anonymous	453650093	12/31/23(Sun)19:00:01	453647428	1704067201	NaN	...
1	https://www.youtube.com/watch?v=8Sb<wbr>UC-UaAxE	GB	United Kingdom	3V/Nw5/s	Anonymous	453650094	12/31/23(Sun)19:00:01	453642900	1704067201	NaN	...
2	>&g...	US	United States	cErt6woZ	Anonymous	453650095	12/31/23(Sun)19:00:01	453647377	1704067201	NaN	...

Figure 6 4Chan Example Documents

	bumplimit	closed	com	com.keyword	country	country.keyword
1	0	-	2024 is looking bright. Based niggas are on our side. BASED.	are on our side. BASED.	US	US
2	0	-	. so anyways This is what a narcissistic personality disorder looks like	onality disorder looks like	-	-
3	0	-	COME DOWN (10hr ver) 8/11/16 https://youtu.be/HcEc8zeNYxI [Embed]	be/HcEc8zeNYxI [Embed]	US	US
4	0	-	ef="/pol/thread/410856566#p410856566" class="quotelink">>>410856566 #	>>410856566 #	US	US
5	0	-	read pol humour thread pol webm thread humor thread humour thread	thread humour thread	US	US
6	0	-	from 2015-2021 about pandering to Niggers, kikes, celebrities, faggots, and trannies?	es, faggots, and trannies?	US	US
7	0	-	Ukraine, at least we don't have to put up with insufferable MIGApedes anymore.	ble MIGApedes anymore.	US	US
8	0	-	or the pede mr dump, trust the plan etc. Why are MIGApedes like this	y are MIGApedes like this	US	US

Figure 7 4Chan President Trump General Ops Example Documents

b. Embedding and Retrieval

Once the texts/documents in the datasets were cleaned, I started generating embeddings in batches. Note that if the document is long, I may have to split the document every n-th occurrence of characters and into passages. For the ToT project, I splitted a document every 1000th occurrence of characters and reorganized the texts in a format like title “is the film name and its description is” passage with a new passage id which is the doc id appended by the number of the passage. Figure 8 shows a splitted corpus which contains 17M records/passages. For the 4Chan project, I skipped such truncation since most 4chan posts have length less than 1000 characters.

	title	text	passage_id
0	Nason airstrip	Nason airstrip is the film name and its descri...	[72614834_1]
1	Elpida Englezou	Elpida Englezou is the film name and its descr...	[72617381_1]
2	Elpida Englezou	Elpida Englezou is the film name and its descr...	[72617381_2]
3	Elpida Englezou	Elpida Englezou is the film name and its descr...	[72617381_3]
4	Montluel station	Montluel station is the film name and its desc...	[72610306_1]
...	...	...	...
17161712	Museum of Work	Museum of Work is the film name and its descri...	[846_4]
17161713	Museum of Work	Museum of Work is the film name and its descri...	[846_5]
17161714	Museum of Work	Museum of Work is the film name and its descri...	[846_6]
17161715	Museum of Work	Museum of Work is the film name and its descri...	[846_7]
17161716	Museum of Work	Museum of Work is the film name and its descri...	[846_8]

Figure 8 Truncated long text in ToT project

For both projects, the encoder and tokenizer I used are DPR Context Encoder and DPR Context Encoder Tokenizer Fast both from pretrained facebook/dpr-ctx_encoder-multiset-base model. For the tokenizer, the parameters I used are texts, paddings, return_tensors, max_length and truncation. The input_ids output from tokenizer becomes the input attribute of encoder along with another parameter return_dict becomes the two input parameters of encoder, which finally generate the embeddings.

Before the retrieval starts, I built a Hugging Face dataset object with a FAISS index to store the output embeddings. Since the dataset can only accept 3 columns: title, text, embeddings, I have to make some minor changes before building a dataset object for the 4Chan project by removing unnecessary columns and renaming the attribute names.

Finally, I used the RagRetriever from the pretrained facebook/rag-sequence-nq model to build a retriever in batches. The parameters I used for the retriever are “facebook/rag-sequence-nq”, index_name, passage_path, index_path, indexed_dataset, output_retrieved, and an optional parameter n_docs. For the tokenizer and model, I used the RagTokenizer and RagSequenceForGeneration both also from the pretrained facebook/rag-sequence-nq.

c. Results and Evaluations

Since I want to not only retrieve the documents/passages but also the associated document/passage scores, I made some changes to customize the output of the model. Specifically, after tokenizing the test queries, I got the attribute input_ids and used it as a parameter of model.question_encoder to generate the embeddings of the test query, this output along with the input_ids and return_tensors become the input parameters for the retriever. The output of the retriever is a dictionary and I named it docs_dict. For the ToT project, the docs_dict has an attribute “doc_ids” containing a sub-attribute passage_id. Thus I used retriever.index.get_doc_dicts(docs_dict["doc_ids"]) [0]['passage_id'] to get the retrieved passage id. For the 4chan project, I used retriever.index.get_doc_dicts(docs_dict["doc_ids"]) [0]['title'] and [0]['text'] to get the retrieved document id and text contents. To get the document score, I used torch.bmm. The input parameters of bmm are the embedded query which is the output of model. question_ encoder along with the attribute retrieved_doc_embeds of doc_dicts.

For the ToT project, I was given a dataset of train queries along with their ids and the correct movie title of the associated query. Thus, this train dataset could be used as my ground truth dataset for evaluation. For the 4Chan project, I was not provided a so-called train dataset so I have to make up one. The way I used to make up a ground truth dataset is a simple keyword search. For example, 4chan posts that contain keywords like ['biden', 'joe biden', 'president biden'] will be collected as the ground truth dataset for Biden-related true queries and posts that contain keywords ['trump', 'donald trump'] as Trump-related true queries, and posts that contains keywords like ['AOC', 'Bernie Sanders', 'Kamala Harris', 'Ron DeSantis', 'Nancy Pelosi', 'Mitch McConnell', 'McConnell', 'Hillary Clinton', 'Obama'] is considered as Other-related true queries. In addition, since there’s no ready-to-use queries for testing, I manually wrote 20 queries for

Biden-related, 2 queries for Trump-related, and 10 queries for other-related, making up to 50 test queries in total. Using January's 4chan posts as my corpus, among the total 2.6M cleaned posts, Trump ground truth dataset contains 11510 posts, Biden ground truth dataset contains 5315 posts, others ground truth dataset contains 4411 posts. They overlap during the counting. Figure 9 shows the retrieved posts along with their associated doc score for the 50 test queries when each query retrieved 10K docs. The ToT project has a required format for results showing attributes 'query_id', 'unused', 'doc_id', 'ranking', 'doc_score', 'run_id' in order as displayed in Figure 10.

	query_id	query	doc_id	doc	doc_score
0	1	How do 4chan users feel about Donald Trump's p...	454760943	starting using 4chan before YouTube stopped ...	83.759331
1	1	How do 4chan users feel about Donald Trump's p...	455031705	The elite are also more responsible than anyon...	72.647713
2	1	How do 4chan users feel about Donald Trump's p...	456336997	Because their entire worldview was developed f...	72.284515
3	1	How do 4chan users feel about Donald Trump's p...	455482887	Just try and assume it to be true. What would ...	76.243217
4	1	How do 4chan users feel about Donald Trump's p...	455061195	So what are the dominant ideas on either side ...	80.119720
...	...	...	...	...	...
499995	50	How has McConnell's tenure as Senate Majority ...	456020169	The great replacement already happened 2,000 y...	61.143745
499996	50	How has McConnell's tenure as Senate Majority ...	455103427	Nikki will win in new hamsphire and go on to b...	65.123314
499997	50	How has McConnell's tenure as Senate Majority ...	455208879	The graph is from a deliberate choice to not b...	66.069565
499998	50	How has McConnell's tenure as Senate Majority ...	454113558	so, witness mentioned him everytime? Nope, j...	69.020294
499999	50	How has McConnell's tenure as Senate Majority ...	453908176	only after the records have been extensively ...	64.974365

Figure 9 4Chan Retrieval Results

```
1  2001  Q0  253850  1   76.7411117553711   baseline
2  2001  Q0  253850  2   76.25492858886719  baseline
3  2001  Q0  5377846 3   76.23931884765625  baseline
4  2001  Q0  57533955  4   75.98313903808594  baseline
5  2001  Q0  2493383 6   75.62542724609375  baseline
6  2001  Q0  14072187  7   75.55545043945312  baseline
7  2001  Q0  63153672  9   75.20948028564453  baseline
8  2001  Q0  1060481 10  75.0637435913086   baseline
9  2001  Q0  53842   11  75.00172424316406  baseline
10 2001  Q0  7675405 12  74.96305847167969  baseline
11 2001  Q0  51547584  13  74.92305755615234  baseline
12 2001  Q0  438077  14  74.90141296386719  baseline
13 2001  Q0  55870035  16  74.78475952148438  baseline
14 2001  Q0  1808725 18  74.74919891357422  baseline
15 2001  Q0  17033927  19  74.74636840820312  baseline
16 2001  Q0  12138277  20  74.68601989746094  baseline
17 2001  Q0  52134721  21  74.6738510131836   baseline
18 2001  Q0  2384840 22  74.66407012939453  baseline
19 2001  Q0  25946   24  74.58242797851562  baseline
20 2001  Q0  1808725 25  74.5556411743164   baseline
```

Figure 10 ToT Results in Submission Format

Regarding the evaluations, I used recall@k the most common metric for both projects. For the ToT project, the recall@1000 is 0.1800 and the recall@180000 is 0.5333. Since the recall ratio increased as the k increased, I think reranking the retrieved outputs may benefit the recall ratio. This is an improvement someone may make in next year's TREC conference. For the 4Chan project, the recall@1000 for Trump-related queries is 0.008055214143897137, the recall@1000

for Biden-related queries is 0.003663609719202042, therecall@1000 for Others-related queries is 0.004575966720242035. These results are all less than 1% which don't make sense. The cause of this might be the low quality of make-up queries and the questionable ground truth I collected.

d. Generation

The ToT project doesn't require the generation part as the final submission is the retrieval output only. For the 4Chan project, I did a few explorations by creating a generator and customizing the generator. Specifically, the default generator I built remains the usage of RagSequenceForGeneration from the pretrained facebook/rag-sequence-nq model taking retriever as a parameter of the generator. The generator/model has an attribute model.generate. The input parameter of model. generate is input_ids which is an attribute of input_dict. Input_dict is the tokenized/encoded query. This is how the default generator is set up. To customize the generator, I would like the generator to take more input parameters, including but not limited to context input ids, context attention, doc scores, num of return sequences, the customized retrieved documents, etc. For example, I have a query "who is the GEOTUS" and I have five customized retrieved documents for this query as following ["Trump is the GEOTUS", "Jesus is the God", "Biden is current President of the US", "Jorden is a US basketball player","Obama is ex-president of the US"]. Their associated scores are [0.9, 0.8, 0.1, 0.1, 0.1]. The generated output from our customized generator shows "Trump" is the generated answer of the query/prompt "who is the GEOTUS", which shows the success of the customized generator.

Future Project Improvements

For the ToT project, the performance might be improved by reranking the retrieved documents. I did a little initial exploration of reranking techniques but not completed well. It would be better for someone to implement a reranking technique from scratch utilizing their own research. The second improvement for the ToT project is adding more metrics for evaluation, particularly adding DCG metric and Precision metric which are the same metrics used by the judges from the committee of ToT project organization. Finally, the performance might be further improved by fine-tuning the DRP model or using a few external dataset.

For the 4Chan project, the most important thing is to re-generate a few reliable ground truth datasets and more powerful queries/prompts for testing. I believe this is the key to get a reasonable recall ratio. Once this challenge was solved, maybe a further improvement on the generation part can be done by introducing some advanced LLM techniques like prompt engineering, etc.

Reference

[1] [2005.11401] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

[2] Guidelines | TREC 2023 Tip-of-the-Tongue (ToT) Track
[3] 2004.04906] Dense Passage Retrieval for Open-Domain Question Answering
[4] [1910.13461] BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension
[5] Attention is All you Need
[6] Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP 2020.